Table 5. Performance of DPOR algorithms on four real programs.

	Traces Explored			Time		
Benchmark	classic	source	optimal	classic	source	optimal
dialyzer	12436	3600	3600	14m46s	5m17s	5m46s
gproc	14080	8328	8104	3m 3s	1m45s	1m57s
poolboy	6018	3120	2680	3m 2s	1m28s	1m20s
rushhour	793375	536118	528984	145m19s	101m55s	105m41s

Table 6. Memory requirements (in MB) for selected benchmarks.

	filesystem(19)	indexer(15)	gproc	rushhour
classic	92.98	245.32	557.31	24.01
source	66.07	165.23	480.96	24.01
optimal	76.17	174.60	481.07	31.07

Performance on real programs. Finally, we evaluate the algorithms on four Erlang applications. The programs are:

dialyzer A parallel static code analyzer [Aronis and Sagonas 2012] included in the Erlang/OTP distribution.

gproc An extended process dictionary (https://github.com/uwiger/gproc).

poolboy A worker pool factory (https://github.com/devinus/poolboy).

rushhour A program that uses processes and ETS tables to solve the Rush Hour puzzle in parallel.

The last program, rushhour, is complex but self-contained (917 lines of code). The first three programs, besides their code, call many modules from the Erlang libraries, which Concuerror also instruments. The total number of lines of instrumented code for testing the first three programs is 44 596, 9 446 and 79 732, respectively.

Table 5 shows the results. Here, the performance differences are not as profound as in synthetic benchmarks. Still, some general conclusions can be drawn:

- Both source-DPOR and optimal-DPOR explore less traces than 'classic' (from 50% up to 3.5 times fewer) and require less time to do so (they run from 42% up to 2.65 times faster).
- Even in real programs, the number of sleep set blocked explorations is significant.
- Regarding the number of traces explored, source-DPOR is quite close to optimal, but manages to completely avoid sleep set blocked executions in only one program (in dialyzer).
- *Source-DPOR* is faster overall, but only slightly so compared to *optimal-DPOR* even though it uses a cheaper test. In fact, its maximal performance difference percentage-wise from *optimal-DPOR* is a bit less than 10% (in dialyzer again).

Although, we do not include a full set of memory consumption measurements, we mention that all algorithms have very similar, and quite low, memory needs. Table 6 shows numbers for gproc, the real program which requires most memory, and for all benchmarks where the difference between source and optimal is more than one MB. From these numbers, it can also be deduced that the size of the wakeup tree is small. In fact, the average size of the wakeup trees for these programs is less than three nodes (it ranges from 2.14 to 2.38 nodes). In our experience, this holds more generally: in all practical applications we have tried, the space requirements of the optimal